

Social Login Setup

This guide explains how to configure OAuth providers like Google so users can sign in to SkillBridge using their existing accounts.

Seed default settings

Run this seed from the `backend` directory to create a default `social_login_settings` row with all providers disabled:

```
knex seed:run --specific=08_social_login_settings_seed.js
```

Google

1. Open the **Google Cloud Console** and create OAuth 2.0 credentials.
2. Add the following URI to the **Authorized redirect URIs** list:

```
http://localhost:5000/api/auth/google/callback
```

Replace `http://localhost:5000` with your production backend URL when deploying. For example:

```
https://yourdomain.com/api/auth/google/callback
```

3. Copy the generated **Client ID** and **Client secret** and enter them in the admin dashboard under **Social Login Settings**. Saving will automatically write the values to `backend/.env`.
4. If the auto-generated redirect URL shown in the admin panel does not match the one configured in Google, enter the desired URL in the **Redirect URL** field for Google and save.
5. Restart the backend so the new credentials take effect.

If the redirect URI does not exactly match what is configured on Google, the login page will display **Error 400: redirect_uri_mismatch**.

If clicking **Sign in with Google** takes you to `/auth/google` and shows a 404 error, verify the frontend's `NEXT_PUBLIC_API_BASE_URL` points to your backend URL including the `/api` prefix. When using Docker Compose, configure this in `frontend/.env.production` and remove or ignore the root `.env` which defaults to `http://backend:5002/api`. Set it to your externally reachable domain such as `https://yourdomain.com/api`. Without this variable the login and register buttons default to `/api/auth/*` which only works when the frontend and backend share the same host. Both buttons now append `?origin=` with the current site origin so the API can redirect back correctly even when `FRONTEND_URL` is not configured.

If requests to `/api/*` still return a Next.js 404 page, confirm your reverse proxy forwards the `/api` path to the backend container. The provided `nginx/conf.d` configs use `location ^~ /api/` to proxy to `backend:5002` without rewriting the prefix. Restart nginx after updating the configuration.

After editing the files you can verify that nginx picked up the change by running:

```
sudo nginx -T | grep '/api'
```

The output should list the `/api` location pointing at the backend. If the block is missing, reload nginx (for example `sudo systemctl reload nginx`) and try the login flow again.

If the authentication completes but you land on a 404 page, check that the backend's `FRONTEND_URL` matches your frontend domain. An incorrect value can cause the API to redirect to the wrong host. You may also omit

`FRONTEND_URL` to use the request's `Origin` header automatically.

GitHub

1. On GitHub open **Settings → Developer settings → OAuth Apps** and create a new OAuth App.
2. Set the **Authorization callback URL** to:

```
http://localhost:5002/api/auth/github/callback
```

Replace `http://localhost:5002` with your backend URL when deployed. For example:

```
https://${APP_DOMAIN}/api/auth/github/callback
```

3. Enter the generated **Client ID** and **Client Secret** in the admin panel under **Social Login Settings**. These values will be stored in `backend/.env` automatically.
4. Save the settings and restart the backend so the GitHub strategy is initialized.

After completing these steps the **Sign in with GitHub** button should redirect back to `/auth/social-success` and automatically log the user in.

Facebook

1. Go to the **Meta for Developers** portal and create a new app with the **Facebook Login** product.
2. Under **Valid OAuth Redirect URIs**, add:

```
https://<backend>/api/auth/facebook/callback
```

Replace `<backend>` with your backend's base URL.

3. Request the `email` and `public_profile` permissions for the app.
4. Enter the generated **App ID** and **App Secret** in the admin panel under **Social Login Settings**. Saving will update `backend/.env`.
5. Restart the backend so the Facebook strategy is initialized.

reCAPTCHA

SkillBridge can optionally validate a Google reCAPTCHA token during login and registration. This behaviour is controlled by the `social_login_settings` record stored in the database. When the `recaptcha.active` flag is `true` the backend will verify the provided token using Google's API before authenticating the user. reCAPTCHA settings are only available under **Social Login Settings** in the admin dashboard.

To temporarily disable reCAPTCHA you can call the configuration endpoint with admin credentials:

```
curl -X PUT http://localhost:5000/api/social-login/config \
-H 'Content-Type: application/json' \
-d '{ "recaptcha": { "active": false } }'
```

Setting `active` back to `true` re-enables the verification check. When disabled, the server only validates the email and password.